

# DevOps Laboratory

## Lab Experiment No. 5

|                                                                                                                                                   |                                                      |
|---------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|
| Name : Vijay Hanumandla                                                                                                                           | Vijay Hanumandla                                     |
| Roll No: 23102C0071                                                                                                                               | 23102C0071                                           |
| Subject: DevOps                                                                                                                                   | DevOps                                               |
| Experiment No : 5                                                                                                                                 | 5                                                    |
| Title: Understand Jenkins Master-Slave Architecture and scale your Jenkins standalone implementation by implementing slave nodes. Implementation: | Jenkins Master-Slave (Controller-Agent) Architecture |

## Aim

To understand the Jenkins Master-Slave (Controller-Agent) Architecture and to scale a standalone Jenkins implementation by configuring and connecting a slave (agent) node.

## Theory

Jenkins is an open-source automation server widely used to implement Continuous Integration and Continuous Deployment (CI/CD) pipelines. In a standalone setup, a single machine runs both the Jenkins Controller (master) and executes all build jobs. This works for small projects, but as the number of jobs grows, the controller machine becomes a bottleneck and a single point of load.

To solve this, Jenkins supports a Master-Slave (Controller-Agent) architecture. In this model, the Jenkins Controller acts as the central coordinator that manages job scheduling, configuration, and the web UI, while one or more Agent (slave) nodes are separate machines that actually execute the build jobs. Agents connect to the controller over the network and register themselves with a name and a label. When a job is configured to run only on a machine with a specific label, the controller forwards that job's execution to the matching agent instead of running it locally.

This architecture allows workloads to be distributed across multiple machines, enables jobs to run on specific environments (for example, a Windows-only or Linux-only agent), and improves the overall scalability and performance of the CI/CD pipeline. The Jenkins Agent program itself runs on Java, so any machine that is to act as an agent must have a compatible JDK installed.

## Implementation

### Step 1: Verify Jenkins on PC 1 (Controller)

On PC 1, Jenkins was started and the browser was pointed to:

```
http://localhost:8080
```

The Jenkins dashboard loaded successfully, confirming that PC 1 is functioning as the Jenkins Controller.

### Step 2: Find the IP address of PC 1

On PC 1's Command Prompt, the following command was run to find the machine's IP address:

```
ipconfig
```

The IPv4 address obtained (e.g., 192.168.1.10) gives the network address at which Jenkins can be reached by other machines:

```
http://192.168.1.10:8080
```

### Step 3: Test connectivity from PC 2

On PC 2, a browser was opened and the above URL (<http://192.168.1.10:8080>) was entered. Jenkins opened successfully on PC 2 as well, confirming that PC 1 and PC 2 can communicate over the network — a necessary precondition for setting up an agent.

### Step 4: Check Java installation on PC 2

Since the Jenkins Agent program requires Java to run, PC 2's Command Prompt was used to verify the Java installation:

```
java -version
```

The installed Java version (e.g., java version "21...") was confirmed, verifying PC 2 was ready to run the agent process.

### Step 5: Create a new Agent node in Jenkins

On the Jenkins Controller (PC 1), a new node was created via:

```
Jenkins Dashboard → Manage Jenkins → Nodes → New Node
```

The node was named "Agent-1" and configured as a Permanent Agent.

### Step 6: Configure the Agent

The following agent configuration was set:

- Number of executors: 1 — allows Agent-1 to run one Jenkins job at a time
- Remote root directory: C:\JenkinsAgent — the working folder Jenkins uses on PC 2
- Label: windows — a category tag so jobs can be restricted to run only on machines carrying this label

### Step 7: Create the working directory on PC 2

On PC 2, the remote root directory was created manually, or using the Command Prompt:

```
mkdir C:\JenkinsAgentcd C:\JenkinsAgent
```

### Step 8: Connect Agent-1 to the Controller

After saving the agent configuration, Jenkins generated a connection command of the form:

```
java -jar agent.jar -url http://192.168.1.10:8080/ -secret XXXXX -name "Agent-1"
```

This command was executed on PC 2. It launches the Jenkins agent process (via Java), which connects to the Jenkins Controller at the given URL using the provided secret, and registers itself under the name "Agent-1". After a successful connection, the Jenkins dashboard showed Agent-1 with an Online status, confirming the Controller-Agent link was established.

### Step 9: Create a Freestyle job restricted to Agent-1

A new Freestyle Project named "AgentDemo" was created via Dashboard → New Item → Freestyle Project. In the job configuration, the option "Restrict where this project can be run" was enabled and the label "windows" was entered. Since Agent-1 was tagged with the same label, Jenkins routes this job specifically to Agent-1 instead of the Controller.

### Step 10: Add a build step and run the job

Under Build Steps → Execute Windows batch command, the following commands were added:

```
echo Hello Studentsecho Jenkins job is running on:hostname
```

The job was saved and triggered using Build Now. Opening the Console Output showed the expected result:

```
Hello StudentsJenkins job is running on:LAB-PC-22
```

Since LAB-PC-22 corresponds to PC 2, this confirmed that the job was executed by the Agent and not by the Controller itself.

## Conclusion

The Jenkins Master-Slave (Controller-Agent) architecture was successfully implemented by configuring PC 1 as the Jenkins Controller and PC 2 as a connected Agent node (Agent-1). A label-restricted Freestyle job (AgentDemo) was created and its console output verified that the build executed on the Agent machine rather than the Controller. This demonstrates how Jenkins can distribute build workloads across multiple machines, which improves scalability and allows jobs to be targeted to specific environments in a CI/CD pipeline.